



机器无关代码优化

Part2: 数据流与到达定值分析

李 诚

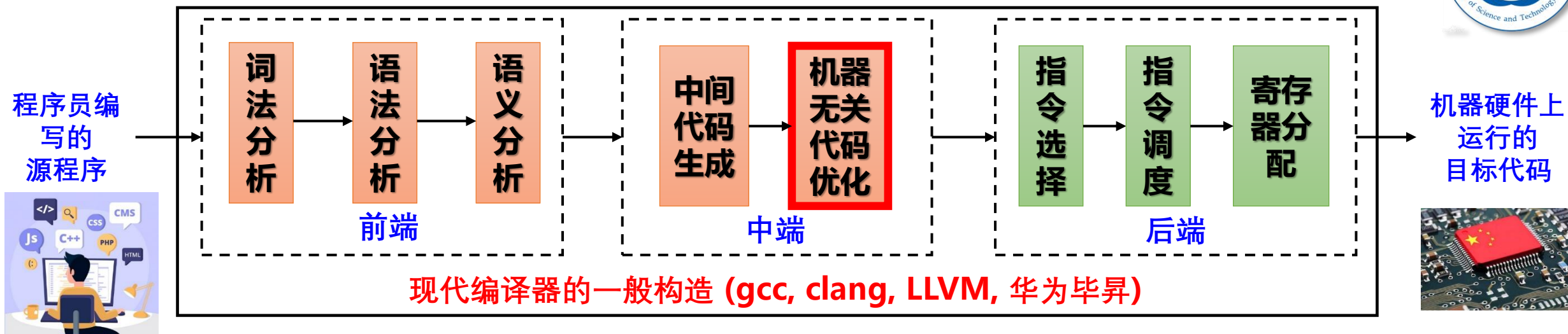
国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2025年11月03日



本节提纲



- 代码优化的主要实现方式
- 数据流分析概述
- 数据流分析理论框架
- 到达-定值分析算法



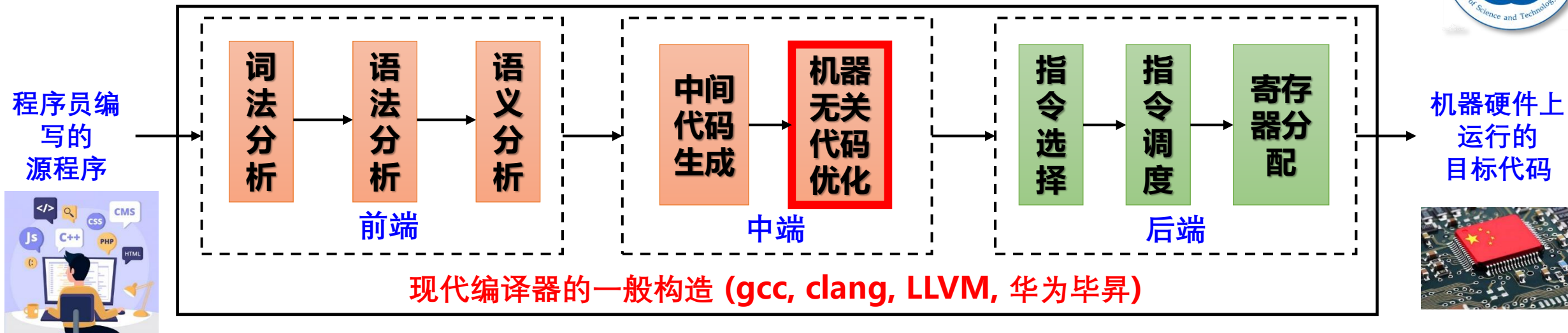
优化的实现方式



- **全局视角-跨基本块的优化**
 - 数据流分析
- **局部视角-基本块的优化**
 - DAG表示



本节提纲



- 代码优化的主要实现方式
- 数据流分析概述
- 数据流分析理论框架
- 到达-定值分析算法



- **Data-flow analysis**

- 一组用来获取程序执行**路径**上的数据流信息的技术

- **数据流分析应用**

- 到达-定值分析(Reaching-Definition Analysis)
 - 活跃变量分析(Live-Variable Analysis)
 - 可用表达式分析(Available-Expression Analysis)
- **在每一种数据流分析应用中，都会把每个**程序点**和一个**数据流值**关联起来**



• 流图上的点(程序点)

- 基本块中，两个相邻的语句之间为程序的一个点
- 基本块的开始点和结束点

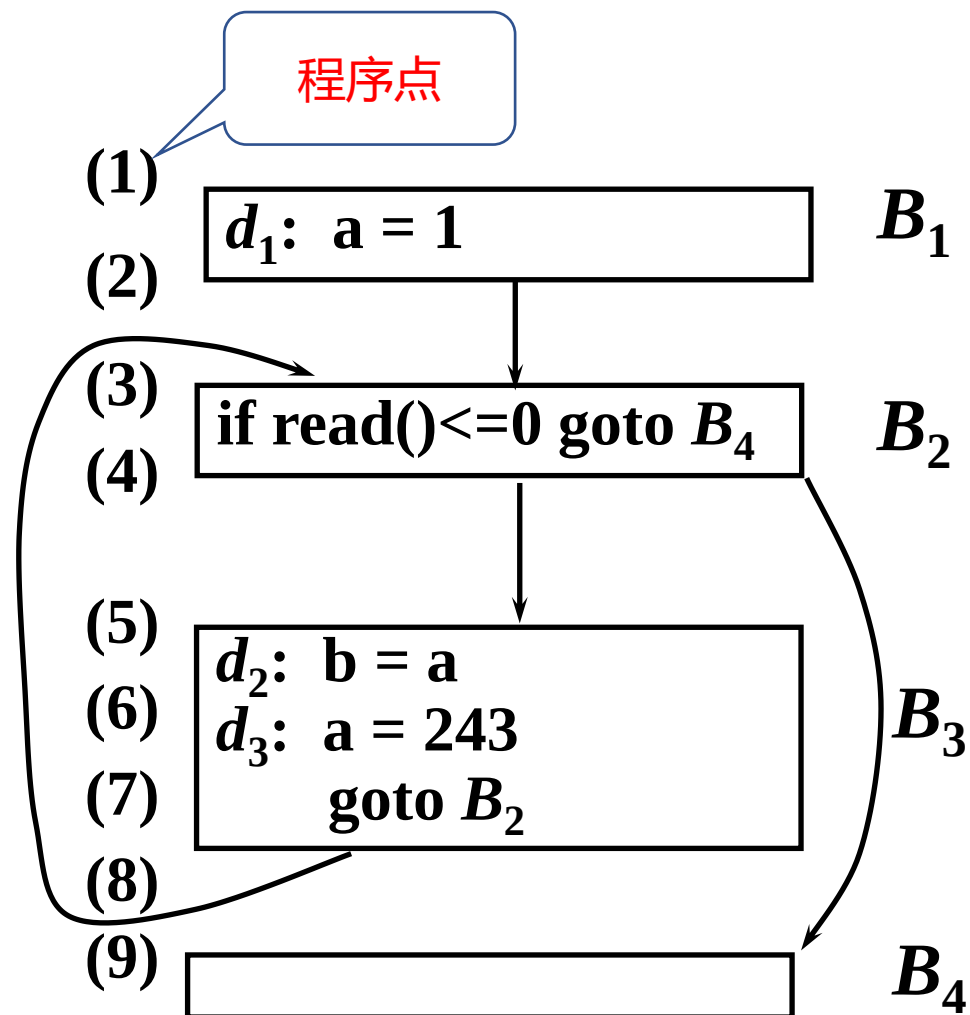
• 流图上的路径

- 点序列 $p_1, p_2, \dots, p_n$ ，对1和 $n - 1$ 间的每个 i ，满足
 - (1) p_i 是先于一个语句的点， p_{i+1} 是同一块中位于该语句后的点，或者
 - (2) p_i 是某块的结束点， p_{i+1} 是后继块的开始点



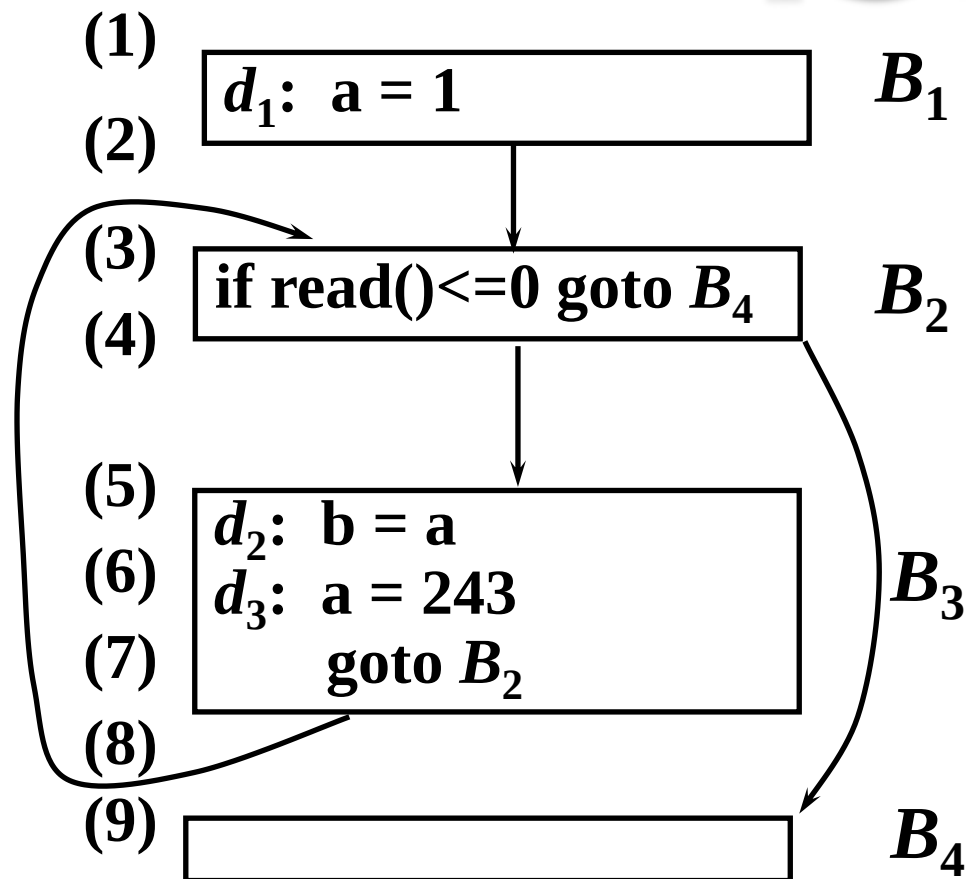
• 流图上路径实例

- (1, 2, 3, 4, 9)
- (1, 2, 3, 4, 5, 6, 7, 8, 3, 4, 9)
- (1, 2, 3, 4, 5, 6, 7, 8, 3, 4, 5, 6, 7, 8, 3, 4, 9)
- (1, 2, 3, 4, 5, 6, 7, 8, 3, 4, 5, 6, 7, 8, 3, 4, 5, 6, 7, 8, ...)
-



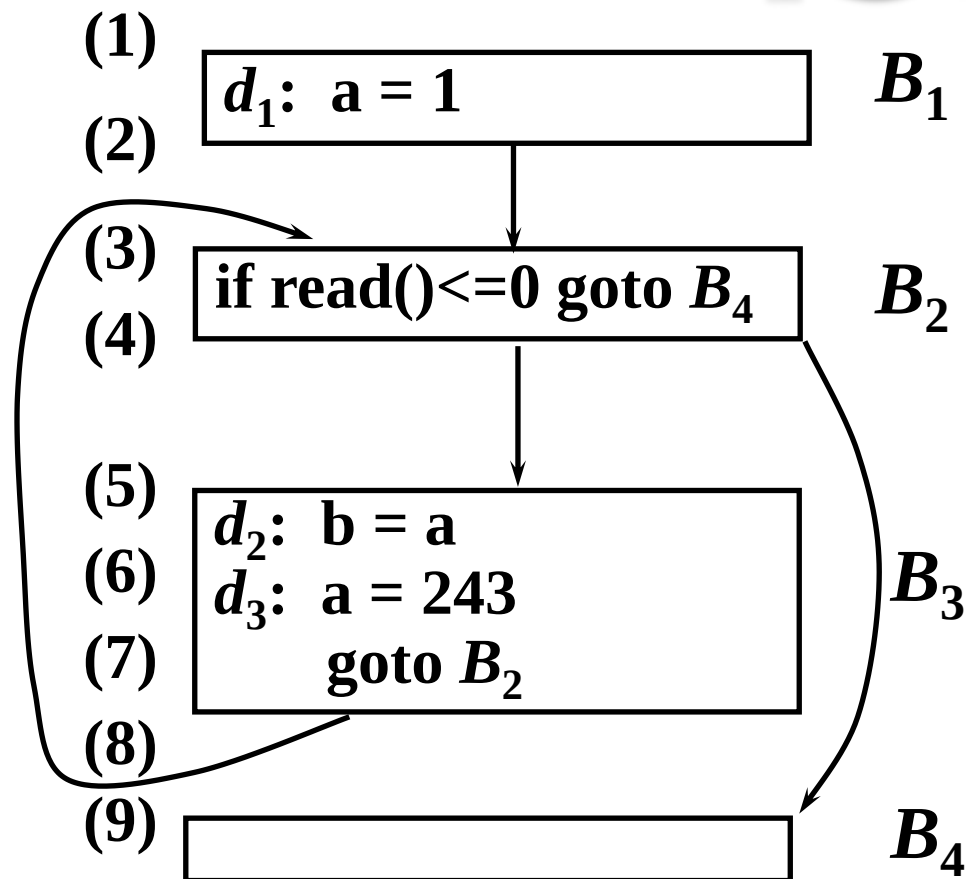


- 分析程序的行为时，必须在其流图上考虑**所有的执行路径**（在调用或返回语句被执行时，还需要考虑执行路径在多个流图之间的跳转）
 - 通常，从流图得到的程序**执行路径数无限**，且执行路径长度没有有限的上界



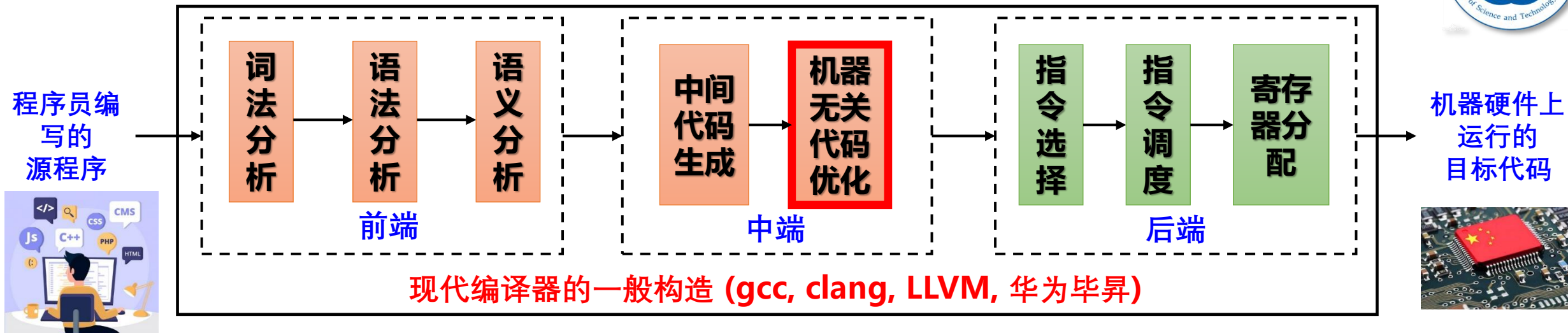


- 分析程序的行为时，必须在其流图上考虑**所有的执行路径**（在调用或返回语句被执行时，还需要考虑执行路径在多个流图之间的跳转）
 - 通常，从流图得到的程序**执行路径数无限**，且执行路径长度没有有限的上界
 - 每个程序点的**不同状态数也可能无限**（程序状态：存储单元到值的映射）





本节提纲



- 代码优化的主要实现方式
- 数据流分析概述
- 数据流分析理论框架
- 到达-定值分析算法



- 数据流值代表在任一程序点能观测到的所有可能程序状态集合的一个**抽象**
- 对于一个语句 s
 - s 之前的程序点对应的数据流值用 $IN[s]$ 表示
 - s 之后的程序点对应的数据流值用 $OUT[s]$ 表示
- 对于一个基本块呢?



- **传递函数(transfer function) f**

- 语句前后两点的数据流值受该语句的语义约束
- 若沿执行路径正向传播, 则 $OUT[s] = f_s(IN[s])$
- 若沿执行路径逆向传播, 则 $IN[s] = f_s(OUT[s])$

若基本块 B 由语句 $s_1, s_2, \dots, s_n$ 依次组成, 则

- $IN[s_{i+1}] = OUT[s_i], i = 1, 2, \dots, n-1$

考虑的是在语句执行后输入输出之间的变化关系



基本块上的数据流模式



□ $IN[B]$: 紧靠基本块B之前的数据流值

❖ $IN[B] = IN[s_1]$

□ $OUT[B]$: 紧靠基本块B之后的数据流值

❖ $OUT[B] = OUT[s_n]$

□ f_B : 基本块B的传递函数

❖ 前向数据流: $OUT[B] = f_B (IN[B])$

➤ $f_B = f_n \dots f_2 f_1$

❖ 逆向数据流: $IN[B] = f_B (OUT[B])$

➤ $f_B = f_1 \dots f_{n-1} f_n$



基本块间的数据流分析模式



- 控制流约束

- 正向传播

$$IN[B] = \cup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$$

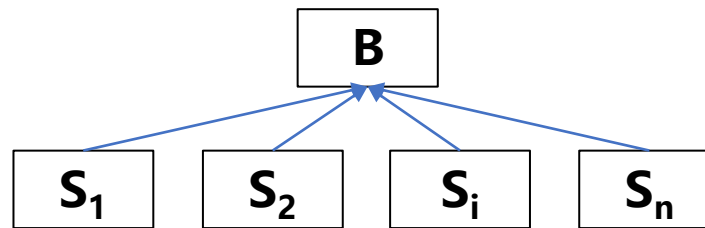
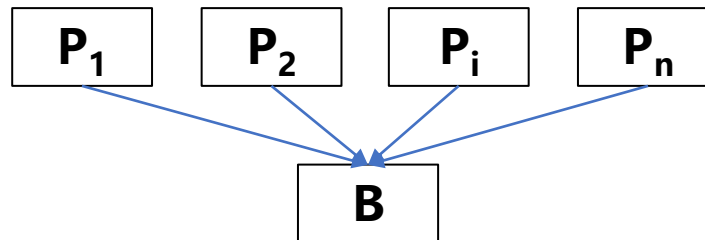
- 逆向传播

$$OUT[B] = \cup_{S \text{ 是 } B \text{ 的后继}} IN[S]$$

- 约束方程组的解通常不是唯一的

- 求解的目标是要找到满足这两组约束（控制流约束和迁移约束）的最“精确”解

U 是汇合的意思，并不一定代表并集，也可能是交集等运算



考虑的是在其他语句或块对于输入的影响和本次执行的输出对其他语句和块的影响



三种有用的分析



- **到达-定值**

- 应用场景：检测未定义的变量

- **可用表达式**

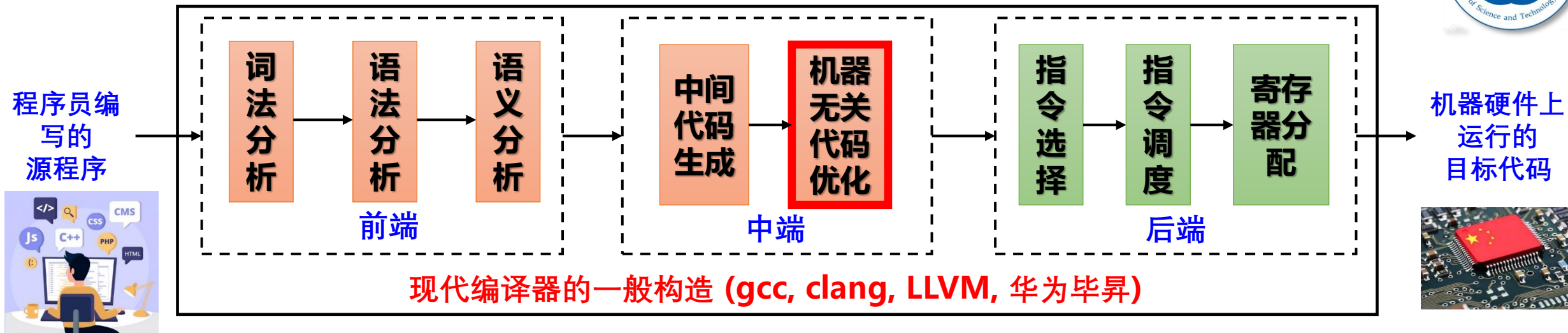
- 应用场景：检测全局公共子表达式

- **活跃变量**

- 应用场景：可用于寄存器分配



本节提纲



- 代码优化的主要实现方式
- 数据流分析概述
- 数据流分析理论框架
- 到达-定值分析算法

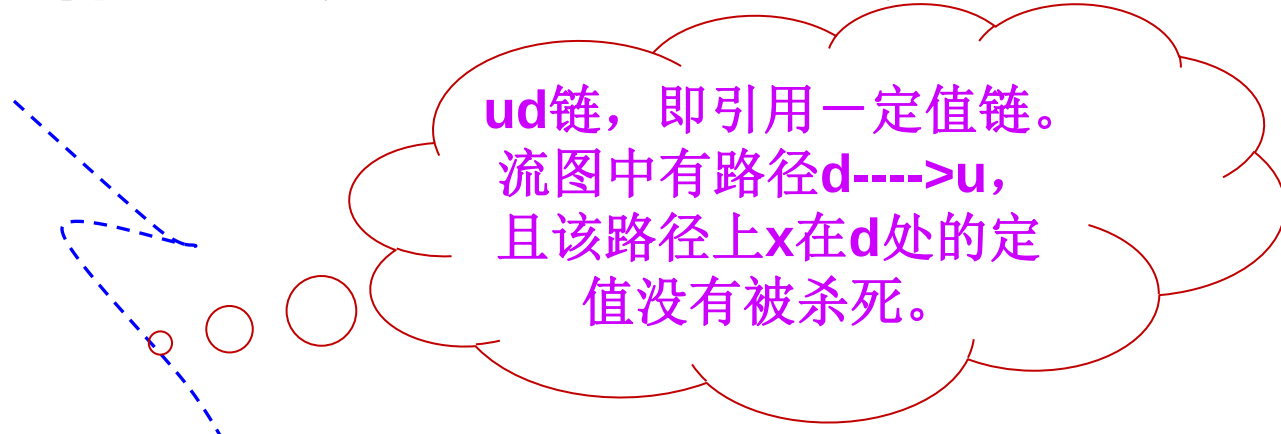


- **到达一个程序点的所有定值(gen)**
- **定值的注销(kill)**
 - 在一条执行路径上，对 x 的赋值注销先前对 x 的所有赋值
- **别名给到达一定值的计算带来困难，因此，本章其余部分仅考虑变量无别名的情况**



• 定值与引用

d : $x := y + z$ // 语句d 是变量x的一个定值点



ud链，即引用一定值链。
流图中有路径**d---->u**，
且该路径上**x**在**d**处的定
值没有被杀死。

u : $w := x + v$ // 语句u 是变量x的一个引用点

• 变量x在d点的定值到达u点



- **循环不变计算的检测**

- 如果循环中含有赋值 $x=y+z$ ，而 y 和 z 所有可能的定值都在循环外，那么 $y+z$ 就是循环不变计算

- **常量合并**

- 如果对变量 x 的某次使用只有一个定值到达，且该定值把一个常量赋给 x ，则可以用该常量替换 x

- **错误检测**

- 判定变量 x 在 p 点上是否未经定值就被引用



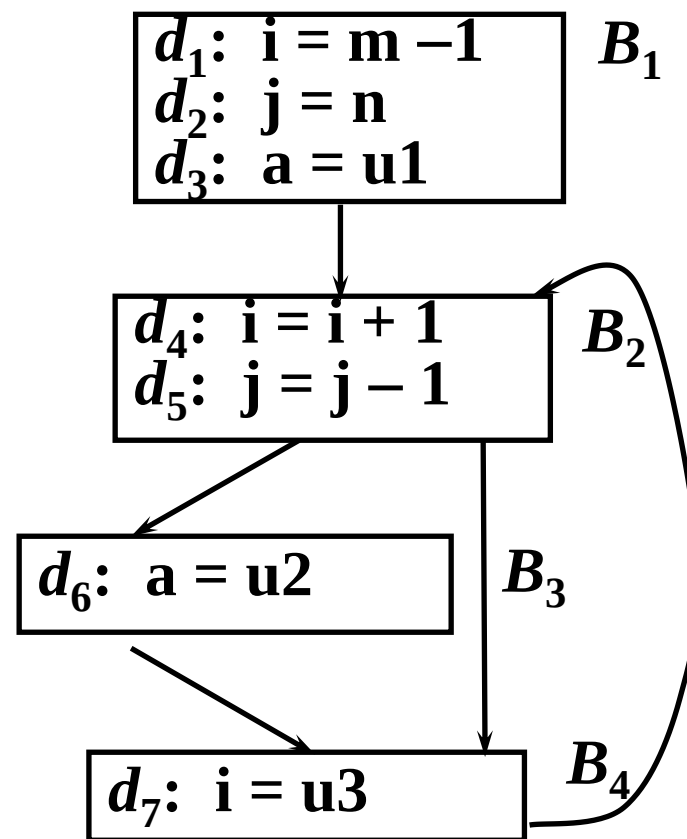
- *gen*和*kill*分别表示一个基本块生成和注销的定值

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$
 $kill [B_3] = \{d_3\}$

$gen [B_4] = \{d_7\}$
 $kill [B_4] = \{d_1, d_4\}$





- 基本块的 gen 和 $kill$ 是怎样计算的

- 对三地址指令 $d: u = v + w$ ，它的状态传递函数是

$$f_d(x) = gen_d \cup (x - kill_d)$$

- 若： $f_1(x) = gen_1 \cup (x - kill_1)$, $f_2(x) = gen_2 \cup (x - kill_2)$

$$\text{则： } f_2(f_1(x)) = gen_2 \cup (gen_1 \cup (x - kill_1) - kill_2)$$

$$= (gen_2 \cup (gen_1 - kill_2)) \cup (x - (kill_1 \cup kill_2))$$

- 若基本块 B 有 n 条三地址指令

$$f_B(x) = gen_B \cup (x - kill_B)$$

$$kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n$$

$$gen_B = gen_n \cup (gen_{n-1} - kill_n) \cup (gen_{n-2} - kill_{n-1} - kill_n) \cup \dots \cup (gen_1 - kill_2 - kill_3 - \dots - kill_n)$$



• 到达一定值的数据流等式

- gen_B : B 中能到达 B 的结束点的定值语句
- $kill_B$: 整个程序中决不会到达 B 结束点的定值
- $IN[B]$: 能到达 B 的开始点的定值集合
- $OUT[B]$: 能到达 B 的结束点的定值集合

两组等式 (根据 gen 和 $kill$ 定义 IN 和 OUT)

- $IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 - $OUT[B] = gen_B \cup (IN[B] - kill_B)$
 - $OUT[ENTRY] = \emptyset$
- ## • 到达一定值方程组的迭代求解, 最终到达不动点



// 正向数据流分析

引入两个虚拟块：ENTRY、EXIT

- (1) $OUT[ENTRY] = \emptyset;$
- (2) for (除了ENTRY以外的每个块B) $OUT[B] = \emptyset;$
- (3) while (任何一个OUT出现变化){
- (4) for (除了ENTRY以外的每个块B) {
- (5) $IN[B] = \cup_{P \text{ 是 } B \text{ 的前驱}} OUT[P];$
- (6) $OUT[B] = gen_B \cup (IN[B] - kill_B);$
- (7) }}

向量求解：集合并操作使用逻辑或，集合相减使用后者求补再逻辑与



到达一定值分析



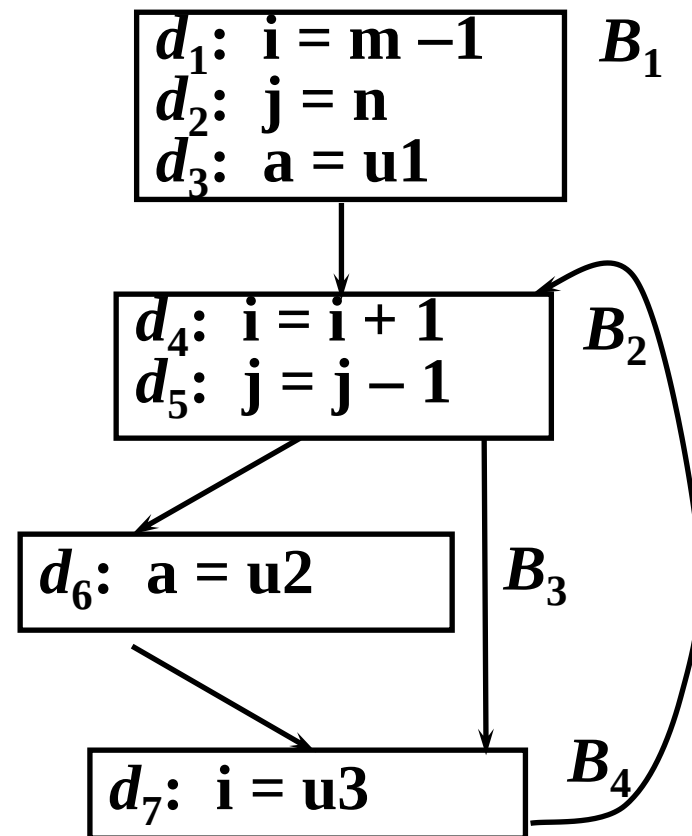
	IN [B]	OUT [B]
B_1		000 0000
B_2		000 0000
B_3		000 0000
B_4		000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \cup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



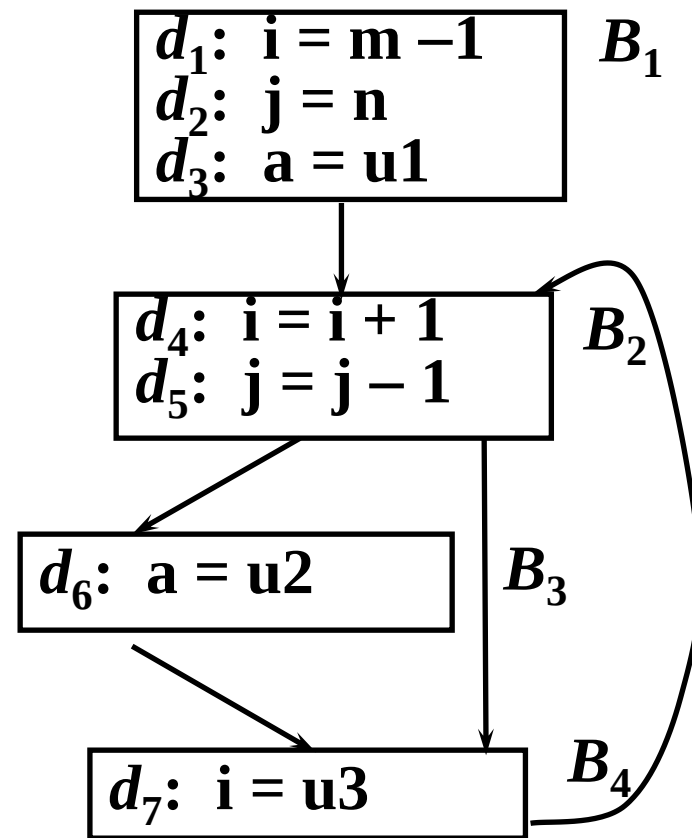
	IN [B]	OUT [B]
B_1	000 0000	000 0000
B_2		000 0000
B_3		000 0000
B_4		000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



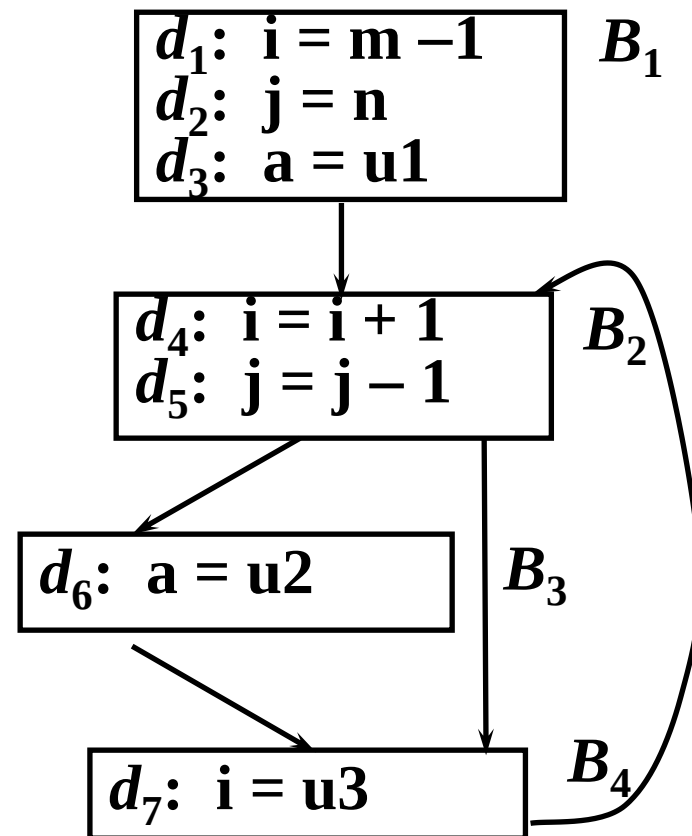
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2		000 0000
B_3		000 0000
B_4		000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



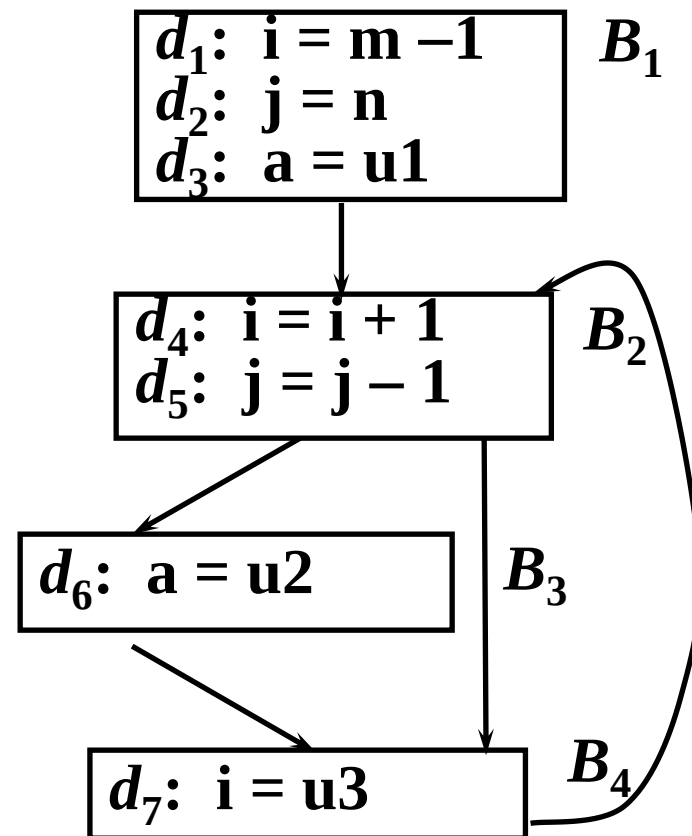
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0000	000 0000
B_3		000 0000
B_4		000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



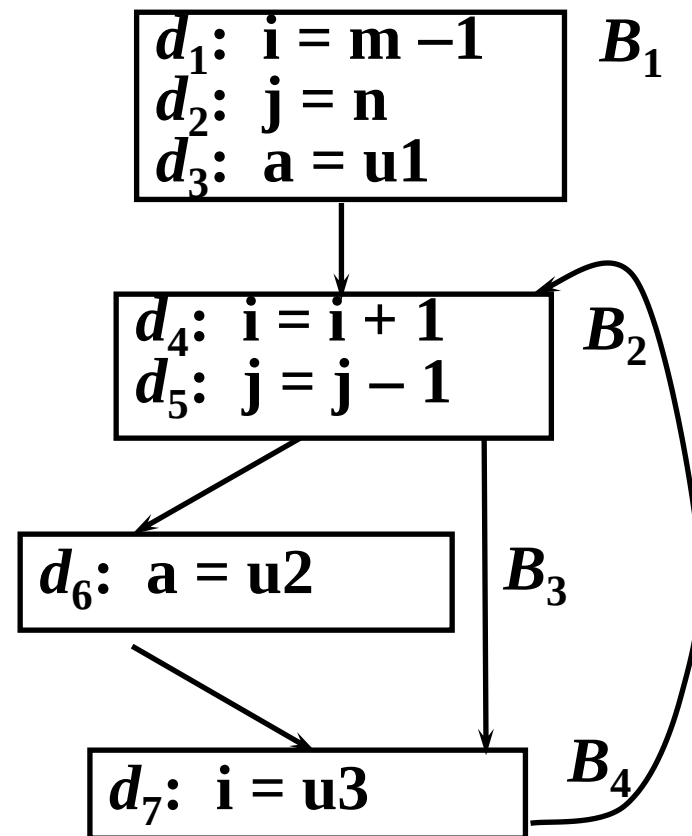
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0000	001 1100
B_3		000 0000
B_4		000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



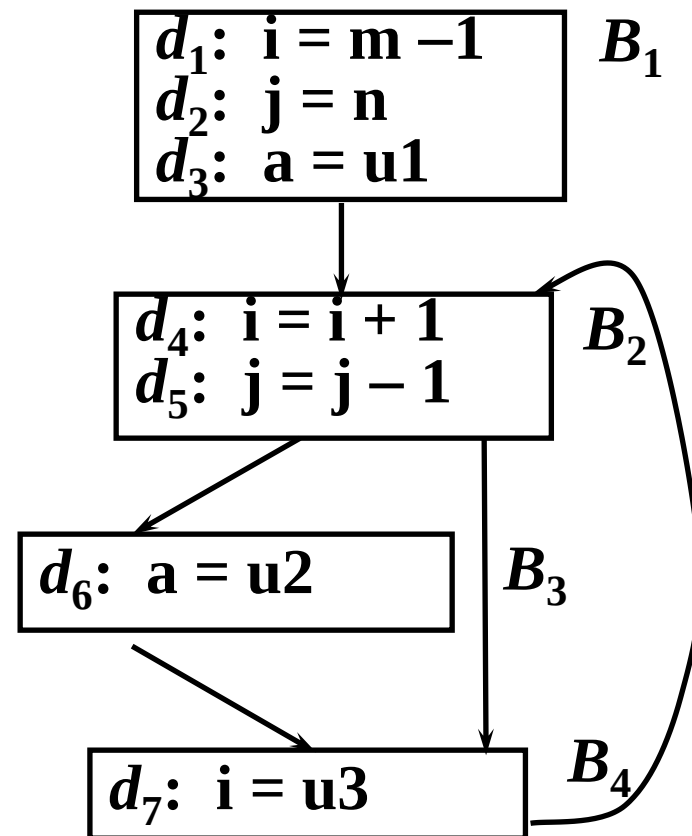
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0000	001 1100
B_3	001 1100	000 0000
B_4		000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



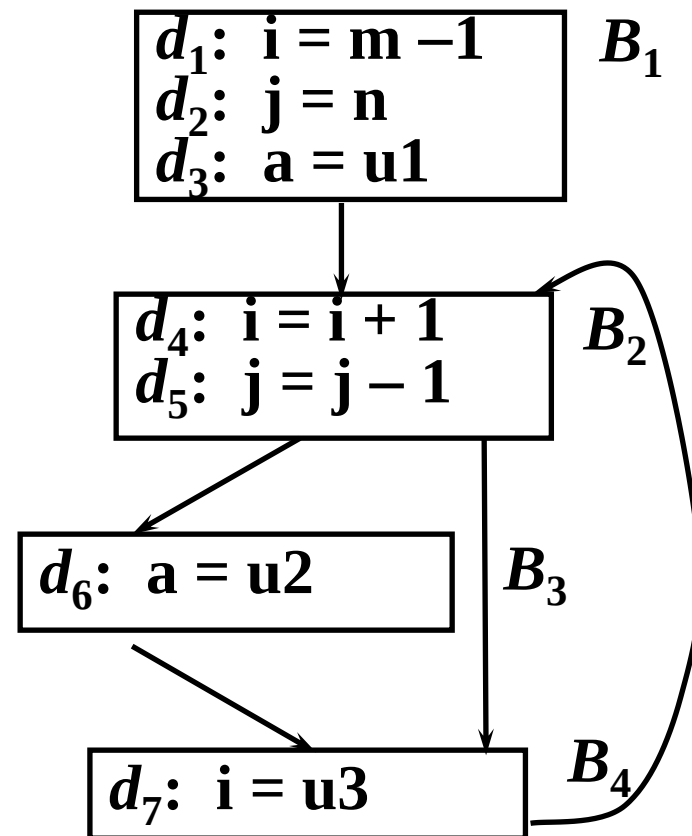
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0000	001 1100
B_3	001 1100	000 1110
B_4		000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \cup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



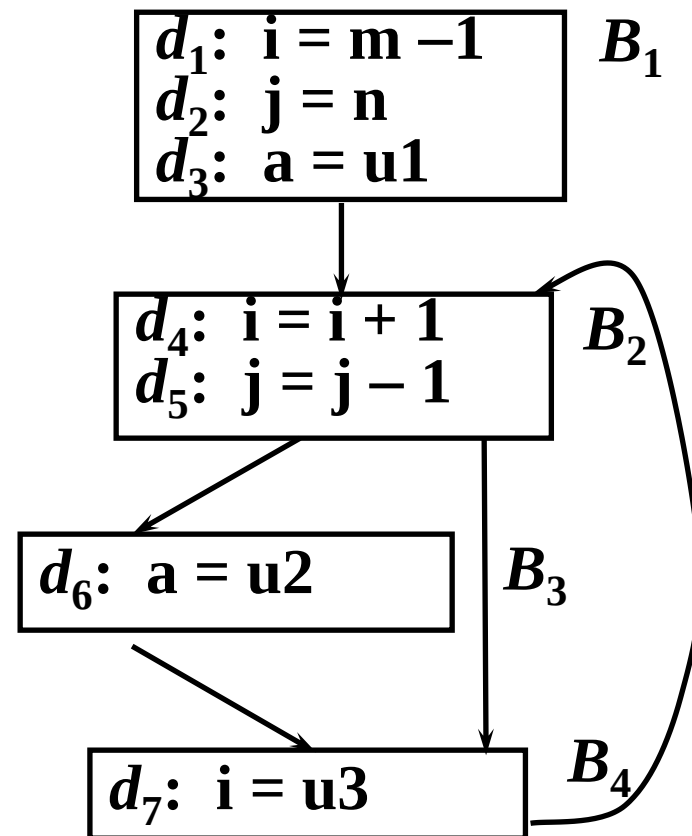
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0000	001 1100
B_3	001 1100	000 1110
B_4	001 1110	000 0000

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



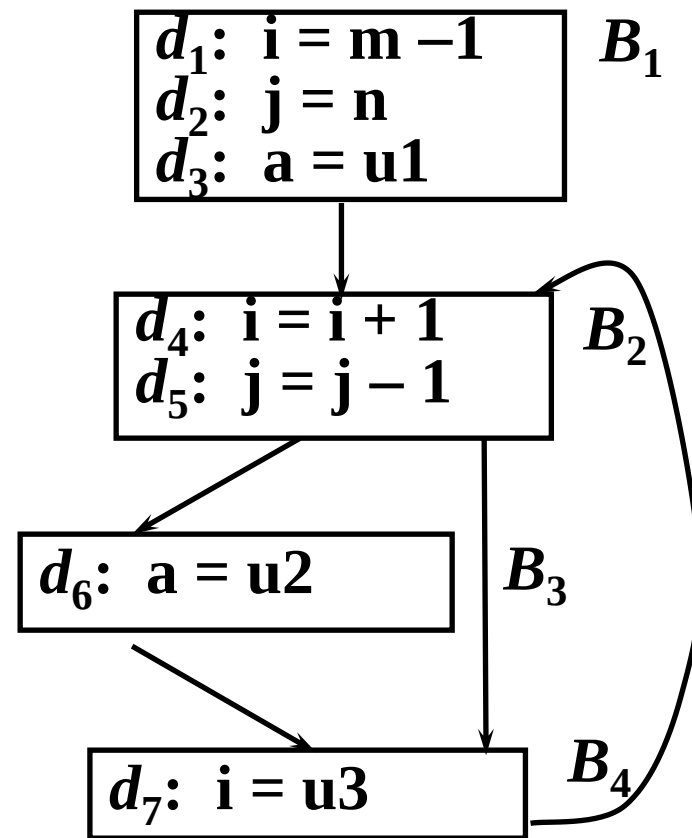
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0000	001 1100
B_3	001 1100	000 1110
B_4	001 1110	001 0111

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



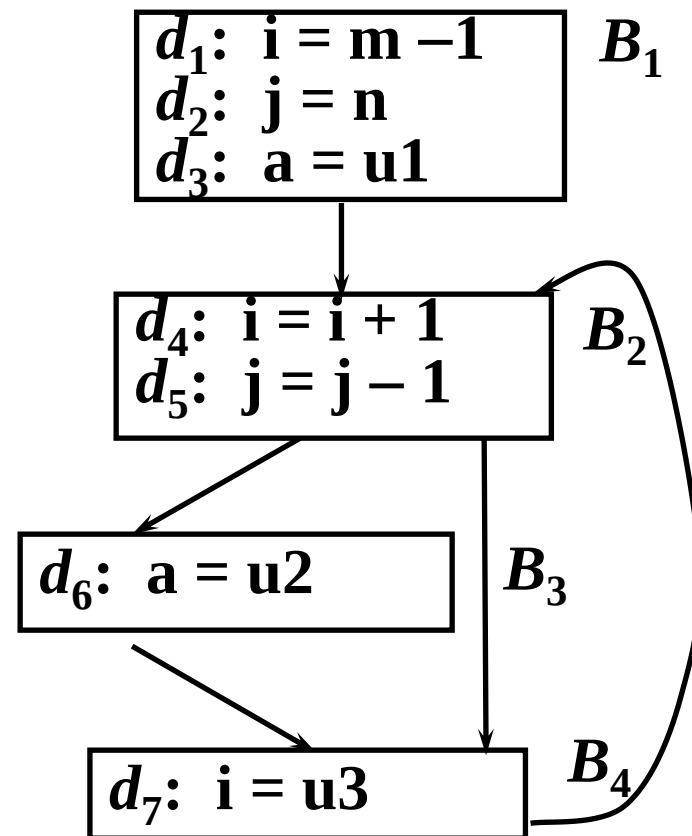
	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0111	001 1100
B_3	001 1100	000 1110
B_4	001 1110	001 0111

$gen [B_1] = \{d_1, d_2, d_3\}$
 $kill [B_1] = \{d_4, d_5, d_6, d_7\}$

$gen [B_2] = \{d_4, d_5\}$
 $kill [B_2] = \{d_1, d_2, d_7\}$

$gen [B_3] = \{d_6\}$ $gen [B_4] = \{d_7\}$
 $kill [B_3] = \{d_3\}$ $kill [B_4] = \{d_1, d_4\}$

$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} OUT[P]$
 $OUT[B] = gen_B \cup (IN[B] - kill_B)$





到达一定值分析



	IN [B]	OUT [B]
B_1	000 0000	111 0000
B_2	111 0111	001 1110
B_3	001 1100	000 1110
B_4	001 1110	001 0111

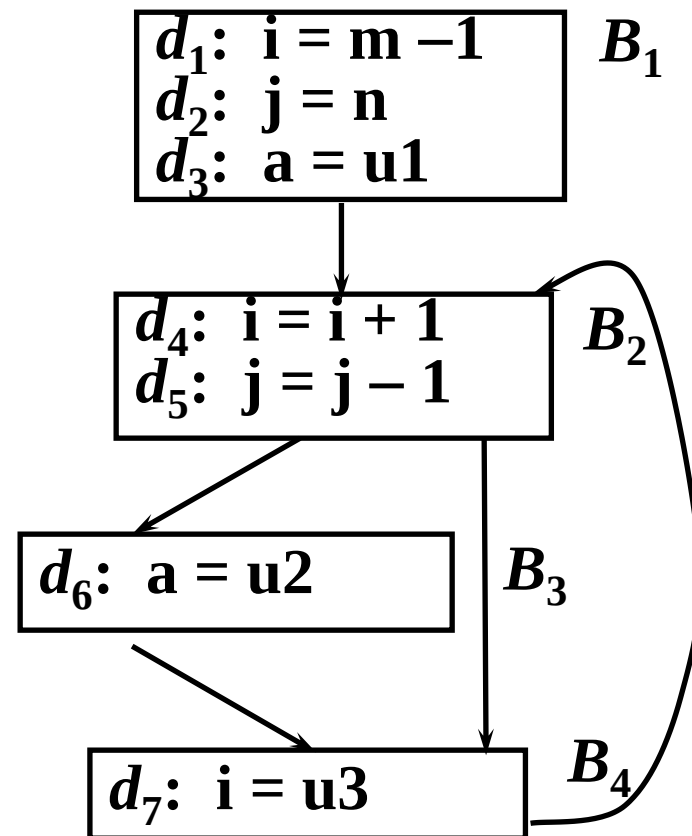
不再继续演示迭代计算

$$\begin{aligned} \text{gen } [B_1] &= \{d_1, d_2, d_3\} \\ \text{kill } [B_1] &= \{d_4, d_5, d_6, d_7\} \end{aligned}$$

$$\begin{aligned} \text{gen } [B_2] &= \{d_4, d_5\} \\ \text{kill } [B_2] &= \{d_1, d_2, d_7\} \end{aligned}$$

$$\begin{aligned} \text{gen } [B_3] &= \{d_6\} & \text{gen } [B_4] &= \{d_7\} \\ \text{kill } [B_3] &= \{d_3\} & \text{kill } [B_4] &= \{d_1, d_4\} \end{aligned}$$

$$\begin{aligned} \text{IN}[B] &= \bigcup_{P \text{ 是 } B \text{ 的前驱}} \text{OUT}[P] \\ \text{OUT}[B] &= \text{gen}_B \cup (\text{IN}[B] - \text{kill}_B) \end{aligned}$$





到达-定值分析非向量计算方法



- 迭代计算

- 计算次序, 深度优先序, 即 $B1 \rightarrow B2 \rightarrow B3 \rightarrow B4$
- 初始值: for all B: $IN[B] = \emptyset$; $OUT[B] = GEN[B]$
- 第一次迭代:

$IN[B1] = \emptyset$; // B1 无前驱结点

$OUT[B1] = GEN[B1] \cup (IN[B1] - KILL[B1]) = GEN[B1] = \{ d1, d2, d3 \}$

$IN[B2] = OUT[B1] \cup OUT[B4] = \{ d1, d2, d3 \} \cup \{ d7 \} = \{ d1, d2, d3, d7 \}$

$OUT[B2] = GEN[B2] \cup (IN[B2] - KILL[B2]) = \{ d4, d5 \} \cup \{ d3 \} = \{ d3, d4, d5 \}$

$IN[B3] = OUT[B2] = \{ d3, d4, d5 \}$

$OUT[B3] = \{ d6 \} \cup (\{ d3, d4, d5 \} - \{ d3 \}) = \{ d4, d5, d6 \}$

$IN[B4] = OUT[B3] \cup OUT[B2] = \{ d3, d4, d5, d6 \}$

$OUT[B4] = \{ d7 \} \cup (\{ d3, d4, d5, d6 \} - \{ d1, d4 \}) = \{ d3, d5, d6, d7 \}$



到达-定值分析非向量计算方法



— 第二次迭代

$IN[B1] = \emptyset$; // B1 无前驱结点

$OUT[B1] = GEN[B1] \cup (IN[B1] - KILL[B1]) = GEN[B1] = \{ d1, d2, d3 \}$

$IN[B2] = OUT[B1] \cup OUT[B4] = \{ d1, d2, d3 \} \cup \{ d3, d5, d6, d7 \} = \{ d1, d2, d3, d5, d6, d7 \}$

$OUT[B2] = GEN[B2] \cup (IN[B2] - KILL[B2]) = \{ d4, d5 \} \cup \{ d3, d5, d6 \} = \{ d3, d4, d5, d6 \}$

$IN[B3] = OUT[B2] = \{ d3, d4, d5, d6 \}$

$OUT[B3] = \{ d6 \} \cup (\{ d3, d4, d5, d6 \} - \{ d3 \}) = \{ d4, d5, d6 \}$

$IN[B4] = OUT[B3] \cup OUT[B2] = \{ d3, d4, d5, d6 \}$

$OUT[B4] = \{ d7 \} \cup (\{ d3, d4, d5, d6 \} - \{ d1, d4 \}) = \{ d3, d5, d6, d7 \}$

经过三次迭代后， $IN[B]$ 和 $OUT[B]$ 不再变化。



- 到达一定值数据流等式是正向的方程

$$\text{OUT}[B] = \text{gen}[B] \cup (\text{IN}[B] - \text{kill}[B])$$

$$\text{IN}[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} \text{OUT}[P]$$

某些数据流等式是反向的

- 到达一定值数据流等式的合流运算是求并集

$$\text{IN}[B] = \bigcup_{P \text{ 是 } B \text{ 的前驱}} \text{OUT}[P]$$

某些数据流等式的合流运算是求交集

- 对到达一定值数据流方程，迭代求它的最小解

某些数据流方程可能要求最大解



一起努力 打造国产基础系统软件体系！

李 诚

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2025年11月03日